# Operating Systems: Seng 3207

## Chapter five

## Process Synchronization

(Materials partly taken from Operating System Concepts by Silberschatz, Galvin and Gagne, 2005 – 7th Edition, chapter 1-2)

## Lecture 5: Introduction to Process Synchronization

# Content

- Background
- The Critical-Section Problem
- Two process solutions
- Peterson’s Solution
- Synchronization Hardware
- Mutex Locks
- Semaphores
- Classic Problems of Synchronization
- Monitors
- Synchronization Examples
- Alternative Approaches

# Background

- The cooperating processes in an operating system can execute concurrently, they can be interrupted at any time during their execution, or they may be partially executed.
- During interleaved execution process may communicate to each other in terms of shared data or a shared location in memory.
- Also processes may need to **share the access of a resource** such as a files, tables etc.
- In both cases, the concurrent access of the shared resource may result in inconsistency in data unless the operation of read/ write to the shared resources are not managed centrally and ordered properly.
- In **process synchronization** we will study the consequences of shared access of resources and how to coordinate these access using some kind of operating system level synchronization protocols.

# Race condition

- A situation where several processes access and manipulate the same data concurrently and the outcome of the execution depends on the particular order in which the access takes place, is called a **race condition**
- Depending upon the implementation of counter++ and counter– operation, a **race condition** can occur if the execution of both the processes is not ordered or enforced by operating system.
- Next slide shows you various implantations of counter operation and how does race condition occur.
-

# Race Condition

- **`counter++`** could be implemented as

```
register1 = counter
register1 = register1 + 1
counter = register1
```

- **`counter--`** could be implemented as

```
register2 = counter
register2 = register2 - 1
counter = register2
```

- Consider this execution interleaving with "count = 5" initially:

| | |
|---|---|
| S0: producer execute `register1 = counter` | {register1 = 5} |
| S1: producer execute `register1 = register1 + 1` | {register1 = 6} |
| S2: consumer execute `register2 = counter` | {register2 = 5} |
| S3: consumer execute `register2 = register2 – 1` | {register2 = 4} |
| S4: producer execute `counter = register1` | {counter = 6 } |
| S5: consumer execute `counter = register2` | {counter = 4} |

# Critical Section problem

- Inconsistency in the state of data, files or tables being accessed by the multiple process at the same time can be attributed to a specific section of code which we call as **critical section.**
- **Critical section** is that part of code executing which in concurrent fashion, two process may come in the state of confusion or inconsistency.
- **Critical section problem** is how to allow to enter and to execute critical section code to different cooperating processes, so that they are able to execute this code in synchronized fashion, as well as they maintain the consistency

# Solution to critical section problem

- In general only one process should be allowed to execute the critical section code.
- Operating system shall enforce a protocol that every participating process should follow in order to ***enter*** and ***exit*** from the critical section.
- A simple solution to manage the entry of single process in critical segment of code is to manage a variable turn which takes two values, one if a process has its turn to execute the critical section code otherwise zero.

# Critical Section

- General structure of process $P_i$

```
do {
    entry section
        critical section
    exit section
        remainder section
} while (true);
```

# Algorithm for Process $P_i$

```
do {

      while (turn == j);

              critical section

      turn = j;

              remainder section

 } while (true);
```

# Properties of a valid Solution to Critical-Section Problem

Consider system of ***n*** processes $\{p_0, p_1, \ldots p_{n-1}\}$ then we define formal solution to the critical section must provide the following guarantees:

1. **Mutual Exclusion -** If process $P_i$ is executing in its critical section, then no other processes can be executing in their critical sections
2. **Progress -** If no process is executing in its critical section and there exist some processes that wish to enter their critical section, then the selection of the processes that will enter the critical section next cannot be postponed indefinitely No assumption concerning **relative speed** of the ***n*** processes

# Critical-Section Handling in OS

3. **Bounded Waiting** - A bound must exist on the number of times that other processes are allowed to enter their critical sections after a process has made a request to enter its critical section and before that request is granted
   - Assume that each process executes at a nonzero speed

4. **No preemption:**

Two approaches depending on if kernel is preemptive or non-preemptive

- **Preemptive** – allows preemption of process when running in kernel mode
- **Non-preemptive** – runs until exits kernel mode, blocks, or voluntarily yields CPU
  - Essentially free of race conditions in kernel mode

# Peterson's Solution

This solution provide an algorithmic description of solving the problem of critical section, this is a two process solution.

- Assume that the **load** and **store** machine-language instructions are atomic; that is, cannot be interrupted
- The two processes share two variables:
  - **int turn;**
  - **Boolean flag[2]**
- The variable **turn** indicates whose turn it is to enter the critical section
- The **flag** array is used to indicate if a process is ready to enter the critical section. **flag[i] =** ***true*** implies that process $\mathbf{P_i}$ is ready!

# Algorithm

- To enter the critical section, process *Pi* first sets flag[i] to be true and then sets turn to the value j, thereby asserting that if the other process wishes to enter the critical section, it can do so.
- If both processes try to enter at the same time, turn will be set to both i and j at roughly the same time. Only one of these assignments will last; the other will occur but will be overwritten immediately.
- The eventual value of turn determines which of the two processes is allowed to enter its critical section first.

# Algorithm for Process $P_i$

```
do {
    flag[i] = true;
    turn = j;
    while (flag[j] && turn = = j);
            critical section
    flag[i] = false;

remainder section
    } while (true);
```

# Both processes code

$P_i$ turn flag F F $P_j$

```
do
{
  flag[i] = true;
  turn = j;
  while (flag[j] && turn == j);

      Critical Section

  flag[i] = false;

      Remainder Section
} while(1)
```

```
do
{
  flag[j] = true;
  turn = i;
  while (flag[i] && turn == i);

      Critical Section

  flag[j] = false;

      Remainder Section
} while(1)
```

Mutual Exclusion

Progress

Activate Windows
Go to Settings to activate Windows

# Cross check the guarantees

- In the Peterson's Algorithm it can be Proved that the three Critical Section requirement are fulfilled by this algorithm:
  1. **Mutual exclusion is preserved**

     $\mathbf{P_i}$ enters CS only if:

     either **flag[j] = false** or **turn = i**
  2. **Progress** requirement is satisfied
  3. **Bounded-waiting** requirement is met

**Limitation:** All the algorithms discussed till now don't work for N process critical section problem. That's why we design hardware based solutions to guarantee n process synchronization.

# Synchronization Hardware

Alternatively, Many systems provide hardware support for implementing the critical section code. Hardware level synchronization is based on the idea of **locking. i.e.** Protecting critical regions via locks.

**Uniprocessor Systems** – The uniprocessor systems can implement locks by disabling the interrupts.

- Currently running code would execute without preemption
- Generally too inefficient on multiprocessor systems, because disabling interrupts on multiprocessros is time consuming and may decrease system efficiency

As a result of this, operating systems using this technique is not broadly scalable.

# Atomic instructions

- Modern machines provide special atomic hardware instructions
    - **Atomic** = non-interruptible

These atomic instructions either test and set the value of a memory word, or they can swap the content of two memory locations without being interrupted.

# Test_and_set Instruction

Definition:

```
boolean test_and_set (boolean *target)
{
     boolean rv = *target;
     *target = TRUE;
     return rv:
}
```

1. Executed atomically
2. Returns the original value of passed parameter
3. Set the new value of passed parameter to “TRUE”.

# Solution to critical section problem using test and set

- The solution to critical section problem can be provided by using test and set instruction
- The process which executes the test and set instruction will hold the lock by initializing a Boolean lock variable to false.
- Since the memory being shared is locked by a process, no other process can enter the CS as the instruction is atomic.

# Solution using testandset

boolean lock = FALSE;

| P1 | P2 | P3 | P4 |
|---|---|---|---|

```
do
{
        while ( TestAndSet(lock) );

                Critical Section

        lock = FALSE;

                Remainder Section

} while(1)
```

```
TestAndSet (lock)
{
        temp = lock;
        lock = true;
        return temp;
}
```

Mutual Exclusion ✔

Progress ✔

Bounded Waiting ✘

lock = FALSE

lock = TRUE

Activate Windows
Go to Settings to activate Windows

# Solution to critical section problem using compare and swap

- Compare and swap operation is atomic in nature and can be used to implement a solution of critical section problem.
- **Mutual exclusion can be provided as follows**: a global variable (lock) is declared and is initialized to 0. The first process that invokes compare and swap() will set lock to 1. It will then enter its critical section, because the original value of lock was equal to the expected value of 0. Subsequent calls to compare and swap() will not succeed, because lock now is not equal to the expected value of 0. When a process exits its critical section, it sets lock back to 0, which allows another process to enter its critical section. The structure of process *Pi* is shown in Figure (5.6 page no. 211 Galvin)
- This method do not satisfy bounded wait condition, next we provide a solution with test and set instruction that satisfies all the CS conditions.

# Solution

boolean lock = FALSE;

```
do
{
        key = TRUE;
        while ( key = TRUE )
                Swap (lock, key);

                Critical Section

        lock = FALSE;

                Remainder Section

} while(1)
```

```
void Swap(a, b)
{
        temp = a;
        a = b;
        b = temp;
}
```

Mutual Exclusion ✔

Progress ✔

Activate Windows
Go to Settings to activate Windo

# Mutex Locks

- Previous solutions are complicated and generally inaccessible to application programmers
- OS designers build software tools to solve critical section problem
- Simplest is mutex lock
- Protect a critical section by first **acquire()** a lock then **release()** the lock
  - Boolean variable indicating if lock is available or not
- Calls to **acquire()** and **release()** must be atomic
  - Usually implemented via hardware atomic instructions
- But this solution requires **busy waiting**
  - This lock therefore called a **spinlock**

# Semaphore

- A semaphore is another way of implementing synchronization among various processes.
- A semaphore is an integer variable S,
- Values of S are used by processes to make decisions to enter the critical section and to wait for other processes to exit their critical section.
- Semaphore uses two functions called wait(S) and Signal(S) in order to communicate the status of critical section among cooperating processes.
- Wait operation simply decrement the current value of S and goes in busy wait loop
- Signal operation increment the value of S by one.

# Semaphore

- Definition of the **wait() operation**

```
wait(S) {
    while (S <= 0)
        ; // busy wait
    S--;
}
```

- Definition of the **signal() operation**

```
signal(S) {
    S++;
}
```

# CS solution using semaphore

- Semaphore integer wait and signal are atomic operations, and if a process modifies the value of semaphore no other process is allowed to edit the value, unless the original process is out of CS.
- Various protocols can be designed using semaphore to implement the solution of critical section problem for cooperating processes.
- Operating systems generally use **Binary Semaphore** to implement synchronization protocol between two processes and a **counting semaphore** for implementing the synchronization among more than two processes.

# Semaphore Usage

- **Counting semaphore** – integer value can range over an unrestricted domain
- **Binary semaphore** – integer value can range only between 0 and 1
  - Same as a **mutex lock**
- Can solve various synchronization problems
- Consider $P_1$ and $P_2$ that require $S_1$ to happen before $S_2$

  Create a semaphore "`synch`" initialized to 0

```
P1:
    S1;
    signal(synch);
P2:
    wait(synch);
    S2;
```

- Can implement a counting semaphore $S$ as a binary semaphore

# Semaphore Implementation

- Must guarantee that no two processes can execute the **wait()** and **signal()** on the same semaphore at the same time
- Thus, the implementation becomes the critical section problem where the **wait** and **signal** code are placed in the critical section
  - Could now have **busy waiting** in critical section implementation
    - But implementation code is short
    - Little busy waiting if critical section rarely occupied
- Note that applications may spend lots of time in critical sections and therefore this is not a good solution

# Semaphore Implementation with no Busy waiting

- With each semaphore there is an associated waiting queue
- Each entry in a waiting queue has two data items:
  - value (of type integer)
  - pointer to next record in the list
- Two operations:
  - **block** – place the process invoking the operation on the appropriate waiting queue
  - **wakeup** – remove one of processes in the waiting queue and place it in the ready queue
- **typedef struct{**

```
int value;
struct process *list;
} semaphore;
```

# Solution using semaphore

mutex

semaphore s = 1;

```
do
{
        wait(s);
                Critical Section
        signal(s);
                Remainder Section
} while(1)
```

```
wait(s)
{
        while(s<=0);
        s--;
}
```

```
signal(s)
{
        s++;
}
```

# Semaphore with No-Busy waiting

semaphore S = 1;

```
do
{
        wait(S);
                Critical Section
        signal(S);
                Remainder Section
} while(1)
```

value = 1

| | | | | | |
|---|---|---|---|---|---|
| Ready Queue | | | | | |
| List | | | | | |

```
wait(S)
{
        value--;
        if(value<0)
        {
                add process in LIST;
                block();
        }
}
```

```
signal(S)
{
        value++;
        if(value<=0)
        {
                remove process from LIST;
                wakeup(P);
        }
}
```

Activate Windows
Go to Settings to activate Windows

# Deadlock and Starvation

- **Deadlock** – two or more processes are waiting indefinitely for an even that can be caused by only one of the waiting processes
- Let ***S*** and ***Q*** be two semaphores initialized to 1

| $P_0$ | $P_1$ |
|---|---|
| **wait(S);** | **wait(Q);** |
| **wait(Q);** | **wait(S);** |
| **...** | **...** |
| **signal(S);** | **signal(Q);** |
| **signal(Q);** | **signal(S);** |

- **Starvation – indefinite blocking**
  - A process may never be removed from the semaphore queue in which it is suspended
- **Priority Inversion** – Scheduling problem when lower-priority process holds a lock needed by higher-priority process
  - Solved via **priority-inheritance protocol**

# Classical Problems of Synchronization

- Following classical problems are used to test newly-proposed synchronization schemes
  - Bounded-Buffer Problem
  - Producer Consumer
  - Readers and Writers Problem
  - Dining-Philosophers Problem

# Bounded-Buffer Problem

- ***n*** buffers, each can hold one item
- Semaphore **mutex** initialized to the value 1
- Semaphore **full** initialized to the value 0
- Semaphore **empty** initialized to the value n

# Bounded Buffer Problem (Cont.)

- The structure of the producer process

```
do {
      ...
       /* produce an item in next_produced */
      ...
    wait(empty);
    wait(mutex);
       ...
       /* add next produced to the buffer */
       ...
    signal(mutex);
    signal(full);
 } while (true);
```

# Illustration of the problem: Example

- In a Producer Consumer Process suppose, the value of counter is 5 at an instant of time.
- If producer as well as consumer both are allowed to access the buffer and they call counter++ and counter– operations respectively without a predefined order, it may lead to a confusion on the final value of counter
- Counter can have a value of 4, 5 or 6 depending upon who executed the statement first.
- Uncertainty in value of counter variable may cause confusion for both the processes.

# Producer

```
while (true) {
        /* produce an item in next produced */

        while (counter == BUFFER_SIZE) ;
                /* do nothing */
        buffer[in] = next_produced;
        in = (in + 1) % BUFFER_SIZE;
        counter++;
}
```

# Consumer

```
while (true) {
        while (counter == 0)
                ; /* do nothing */
        next_consumed = buffer[out];
        out = (out + 1) % BUFFER_SIZE;
         counter--;
        /* consume the item in next consumed */
}
```

# Bounded Buffer Problem (Cont.)

- The structure of the consumer process

```
Do {
    wait(full);
    wait(mutex);
        ...
    /* remove an item from buffer to next_consumed */
        ...
    signal(mutex);
    signal(empty);
        ...
    /* consume the item in next consumed */
        ...
 } while (true);
```

# Readers-Writers Problem

- A data set is shared among a number of concurrent processes
  - Readers – only read the data set; they do ***not*** perform any updates
  - Writers – can both read and write
- Problem – allow multiple readers to read at the same time
  - Only one single writer can access the shared data at the same time
- Several variations of how readers and writers are considered – all involve some form of priorities
- Shared Data
  - Data set
  - Semaphore **rw_mutex** initialized to 1
  - Semaphore **mutex** initialized to 1
  - Integer **read_count** initialized to 0

# Readers-Writers Problem (Cont.)

- The structure of a writer process

```
do {
      wait(rw_mutex);
         ...
      /* writing is performed */
         ...
   signal(rw_mutex);
} while (true);
```

# Readers-Writers Problem (Cont.)

- The structure of a reader process

```
do {
        wait(mutex);
        read_count++;
        if (read_count == 1)
        wait(rw_mutex);
     signal(mutex);
         ...
        /* reading is performed */
         ...
     wait(mutex);
        read count--;
        if (read_count == 0)
     signal(rw_mutex);
     signal(mutex);
} while (true);
```

# Readers-Writers Problem Variations

- ***First*** variation – no reader kept waiting unless writer has permission to use shared object
- ***Second*** variation – once writer is ready, it performs the write ASAP
- Both may have starvation leading to even more variations
- Problem is solved on some systems by kernel providing reader-writer locks

# Dining-Philosophers Problem

- Philosophers spend their lives alternating thinking and eating
- Don't interact with their neighbors, occasionally try to pick up 2 chopsticks (one at a time) to eat from bowl
  - Need both to eat, then release both when done
- In the case of 5 philosophers
  - Shared data
    - Bowl of rice (data set)
    - Semaphore chopstick [5] initialized to 1

# Dining-Philosophers Problem Algorithm

- The structure of Philosopher *i*:

```
do {
    wait (chopstick[i] );
     wait (chopStick[ (i + 1) % 5] );

                //  eat

     signal (chopstick[i] );
     signal (chopstick[ (i + 1) % 5] );

                 //  think

} while (TRUE);
```

- What is the problem with this algorithm?

# Dining-Philosophers Problem Algorithm (Cont.)

- Deadlock handling
  - Allow at most 4 philosophers to be sitting simultaneously at the table.
  - Allow a philosopher to pick up the forks only if both are available (picking must be done in a critical section.
  - Use an asymmetric solution -- an odd-numbered philosopher picks up first the left chopstick and then the right chopstick. Even-numbered philosopher picks up first the right chopstick and then the left chopstick.

# Problems with Semaphores

- Incorrect use of semaphore operations:
  - signal (mutex) …. wait (mutex)
  - wait (mutex) … wait (mutex)
  - Omitting of wait (mutex) or signal (mutex) (or both)
- Deadlock and starvation are possible.

# Monitors

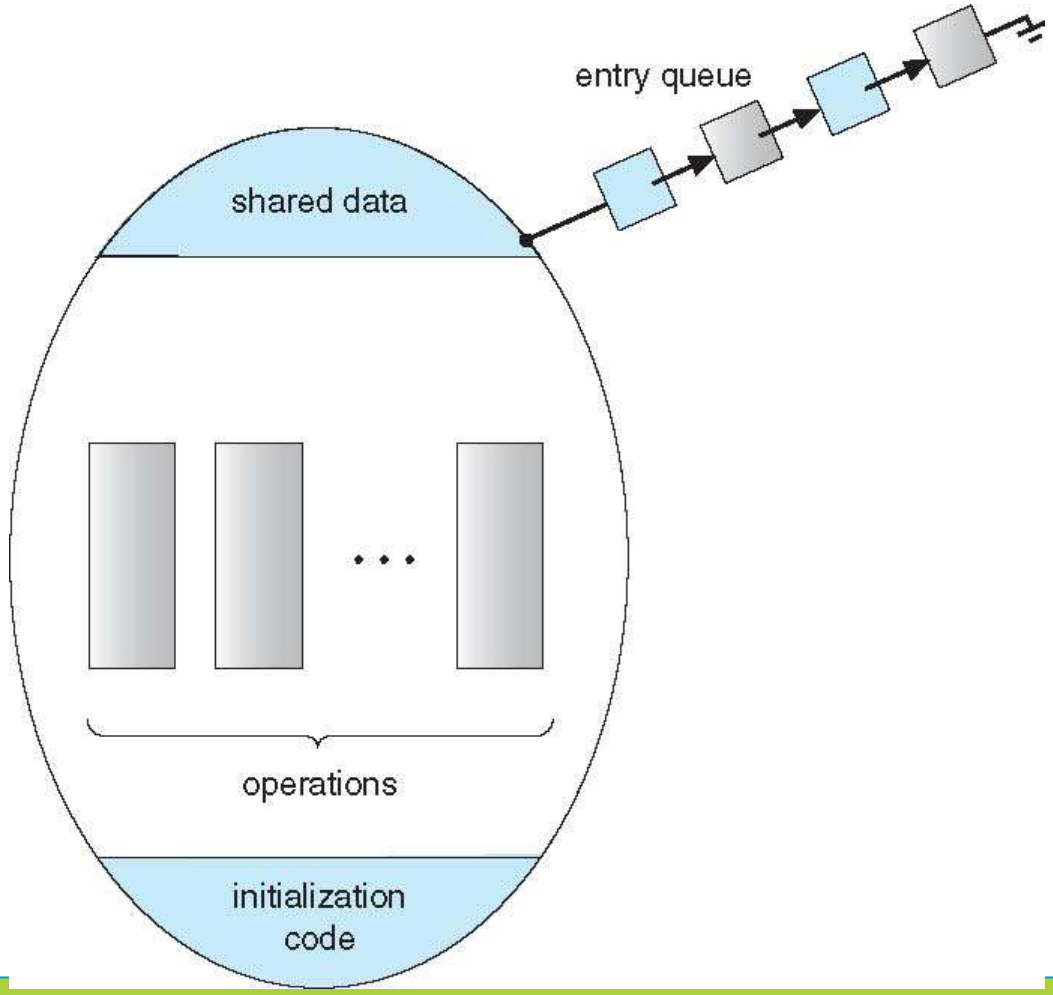
- A high-level abstraction that provides a convenient and effective mechanism for process synchronization
- *Abstract data type*, internal variables only accessible by code within the procedure
- Only one process may be active within the monitor at a time
- But not powerful enough to model some synchronization schemes

```
monitor monitor-name
{
  // shared variable declarations
  procedure P1 (…) { …. }

  procedure Pn (…) {……}

    Initialization code (…) { … }
  }
}
```

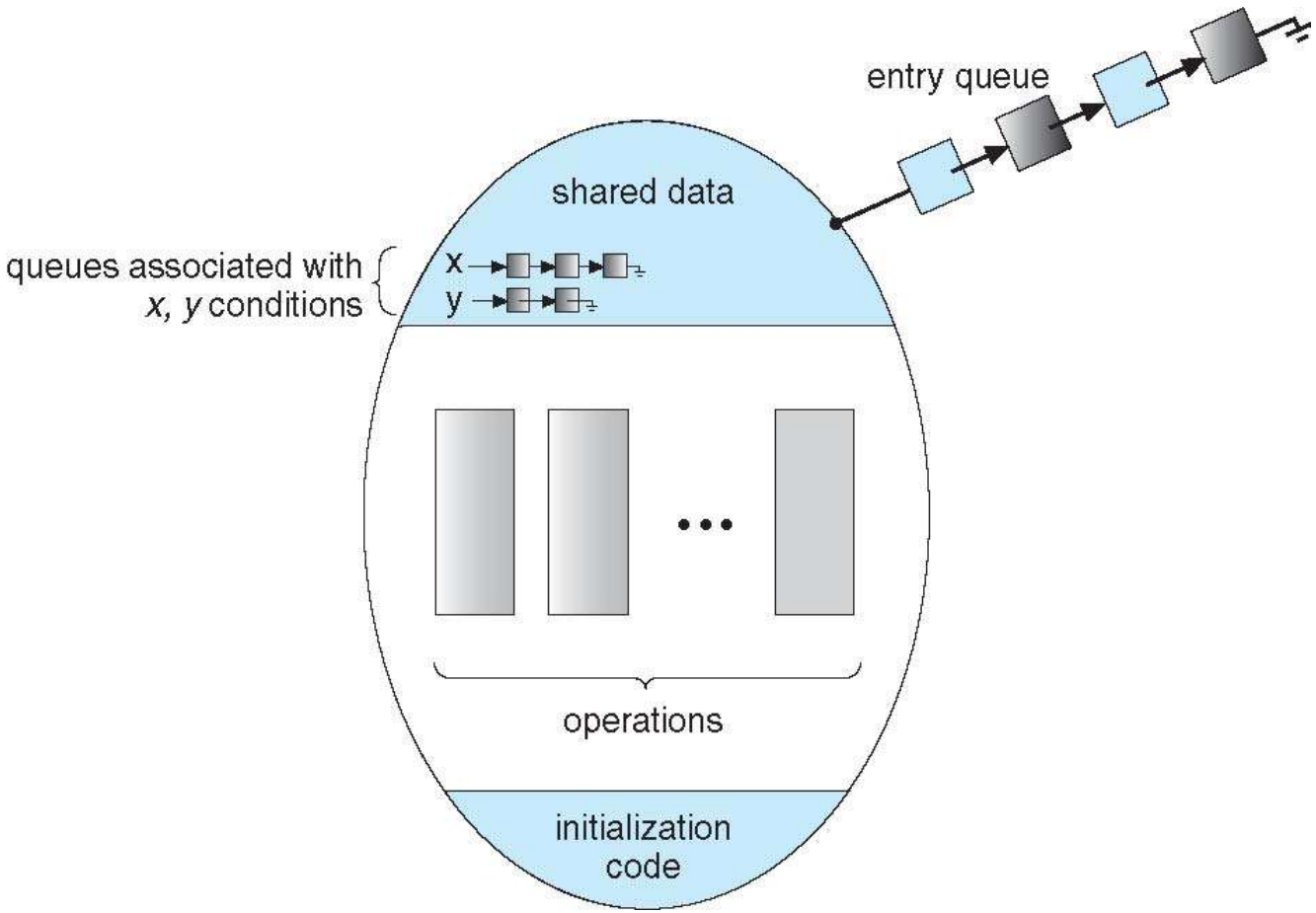
# Schematic view of a Monitor



# Condition Variables

- `condition x, y;`
- Two operations are allowed on a condition variable:
  - `x.wait()` – a process that invokes the operation is suspended until `x.signal()`
  - `x.signal()` – resumes one of processes (if any) that invoked `x.wait()`
    - If no `x.wait()` on the variable, then it has no effect on the variable

# Monitor with Condition Variables



# Condition Variables Choices

- If process P invokes **x.signal(),** and process Q is suspended in **x.wait()**, what should happen next?
  - Both Q and P cannot execute in paralel. If Q is resumed, then P must wait
- Options include
  - **Signal and wait** – P waits until Q either leaves the monitor or it waits for another condition
  - **Signal and continue** – Q waits until P either leaves the monitor or it waits for another condition
  - Both have pros and cons – language implementer can decide
  - Monitors implemented in Concurrent Pascal compromise
    - P executing signal immediately leaves the monitor, Q is resumed
  - Implemented in other languages including Mesa, C#, Java

# Monitor Solution to Dining Philosophers

```
monitor DiningPhilosophers
{
   enum { THINKING; HUNGRY, EATING) state [5] ;
   condition self [5];

   void pickup (int i) {
          state[i] = HUNGRY;
          test(i);
          if (state[i] != EATING) self[i].wait;
}

   void putdown (int i) {
          state[i] = THINKING;
                   // test left and right neighbors
           test((i + 4) % 5);
           test((i + 1) % 5);
}
```

# Solution to Dining Philosophers (Cont.)

```
  void test (int i) {
          if ((state[(i + 4) % 5] != EATING) &&
          (state[i] == HUNGRY) &&
          (state[(i + 1) % 5] != EATING) ) {
               state[i] = EATING ;
           self[i].signal () ;
          }
   }

       initialization_code() {
          for (int i = 0; i < 5; i++)
          state[i] = THINKING;
        }
}
```

# Solution to Dining Philosophers (Cont.)

- Each philosopher *i* invokes the operations **`pickup()`** and **`putdown()`** in the following sequence:

```
DiningPhilosophers.pickup(i);

EAT

DiningPhilosophers.putdown(i);
```

- No deadlock, but starvation is possible

# Monitor Implementation Using Semaphores

- Variables

```
semaphore mutex;  // (initially  = 1)
semaphore next;   // (initially  = 0)
int next_count = 0;
```

- Each procedure ***F*** will be replaced by

```
wait(mutex);
      …
 body of F;
      …
if (next_count > 0)
  signal(next)
else
  signal(mutex);
```

- Mutual exclusion within a monitor is ensured

# Monitor Implementation – Condition Variables

- For each condition variable ***x***, we have:

```
semaphore x_sem; // (initially  = 0)
int x_count = 0;
```

- The operation x.wait can be implemented as:

```
x_count++;
if (next_count > 0)
      signal(next);
else
      signal(mutex);
wait(x_sem);
x_count--;
```

# Monitor Implementation (Cont.)

- The operation **`x.signal`** can be implemented as:

```
if (x_count > 0) {
  next_count++;
  signal(x_sem);
  wait(next);
  next_count--;
}
```

# Resuming Processes within a Monitor

- If several processes queued on condition x, and x.signal() executed, which should be resumed?
- FCFS frequently not adequate
- **conditional-wait** construct of the form x.wait(c)
  - Where c is **priority number**
  - Process with lowest number (highest priority) is scheduled next

# Single Resource allocation

- Allocate a single resource among competing processes using priority numbers that specify the maximum time a process plans to use the resource

```
R.acquire(t);
    ...
  access the resurce;
    ...

R.release;
```

- Where R is an instance of type **ResourceAllocator**

# A Monitor to Allocate Single Resource

```
monitor ResourceAllocator
{
   boolean busy;
   condition x;
   void acquire(int time) {
           if (busy)
              x.wait(time);
           busy = TRUE;
   }
   void release() {
           busy = FALSE;
           x.signal();
   }
initialization code() {
    busy = FALSE;
   }
}
```

# Self reading

- Read About classical synchronization problems from book.
- Implementation of monitors from book in details.
- Write a program for implementation of semaphore in C++ and test that program on two or more processes.